

Módulo 8 – Capítulo 08 – Sección 01

Anatomía de la latencia: prefill, decode y transferencia de datos

El sistema de serving configurado en los capítulos anteriores —vLLM con PagedAttention, continuous batching y la cuantización correcta para el hardware disponible— es la base. Pero incluso con la configuración correcta del motor, la latencia percibida por el usuario puede ser mayor de la esperada, y reducirla requiere entender exactamente de dónde proviene cada milisegundo. La latencia total de una petición de inferencia no es un número monolítico: es la suma de tres fases secuenciales con perfiles de optimización completamente distintos, y atacar la fase incorrecta produce mejoras marginales en lugar de las significativas.

La **fase de prefill** procesa todos los tokens del prompt de entrada en un único forward pass paralelo. A diferencia del decode, el prefill puede procesar todos los N tokens del prompt simultáneamente porque el mecanismo de atención causal en el forward pass de prefill puede computar todas las relaciones de atención entre todos los tokens de entrada en una sola operación de GEMM grande. Esta naturaleza paralela hace que el prefill sea una operación compute-intensive (dominada por la multiplicación matricial) que aprovecha bien los Tensor Cores de las GPUs modernas. El tiempo de prefill crece aproximadamente de forma lineal con el número de tokens del prompt: un prompt de 4.000 tokens toma aproximadamente 4x más que uno de 1.000 tokens en la misma GPU. Flash Attention reduce la constante del crecimiento mediante IO-aware tiling que minimiza las transferencias a VRAM durante el cómputo de atención, pero no cambia la complejidad asintótica.

La **fase de decode** es donde el modelo genera los tokens de respuesta uno a uno en N forward passes secuenciales, usando el KV cache acumulado de los pasos anteriores para no recomputar la atención sobre los tokens ya procesados. A diferencia del prefill, el decode es una operación memory-bound, no compute-bound: en cada paso de decode, la GPU debe leer los pesos completos del modelo desde VRAM para generar un único token. Con un modelo de 7B en BF16 (14 GB de pesos), la GPU debe leer esos 14 GB en cada step de decode. Una A100 con 2 TB/s de ancho de banda de VRAM puede leer 14 GB en aproximadamente 7ms, lo que establece un límite físico de ~ 140 tokens/s para una única request en decode,

independientemente de cuántos TFLOPS tenga la GPU. Incrementar el batch size es la única forma de mejorar la eficiencia del decode: con 16 requests en continuous batching, la misma lectura de los 14 GB de pesos genera 16 tokens simultáneamente, elevando el throughput agregado a ~2.240 tokens/s.

El **crecimiento del KV cache** durante el decode introduce una latencia progresiva que frecuentemente se ignora en los benchmarks: en cada step de decode, el KV cache del token recién generado se añade al pool. Para contextos muy largos (conversaciones de 50+ turnos, documentos de 8K-32K tokens), el KV cache puede ocupar varios GB de VRAM y el acceso a ese cache creciente en cada step de decode consume una proporción creciente del ancho de banda de VRAM, produciendo una degradación gradual del TBT a medida que la conversación avanza. PagedAttention de vLLM mitiga este problema mediante bloques de KV cache no contiguos que no producen fragmentación, y el prefix caching elimina el crecimiento del KV cache para los prefijos compartidos.

Desglose técnico de la latencia de inferencia

- **Prefill (Time to Prefill):** crece lineal con la longitud del contexto; compute-bound; Flash Attention 2/3 reduce la constante; contextos de >4K tokens pueden dominar el TTFT.
- **Decode memory-bound:** el cuello de botella es el ancho de banda de VRAM; A100 a 2 TB/s, 14 GB de pesos → ~7ms por step → ~140 tokens/s máximo por request individual.
- **KV cache growth:** el KV cache crece en cada step de decode; en contextos muy largos, el TBT se degrada progresivamente por mayor presión sobre el ancho de banda de VRAM.
- **Latencia de red y streaming:** patrón SSE o WebSocket envía cada token al cliente inmediatamente; latencia de red por token de 1-5ms en redes de baja latencia; negligible respecto al tiempo de decode.
- **Chunked prefill:** vLLM con `--enable-chunked-prefill` divide prompts largos en chunks intercalados con pasos de decode; reduce la variación del TTFT para requests con prompts de distinta longitud.

Nota del Arquitecto: El indicador más claro de que un sistema tiene problemas de latencia de decode (no de prefill) es cuando el TTFT es aceptable pero los usuarios se quejan de que el texto "aparece muy lento" después del primer token. Ese síntoma indica que el TBT está por encima del umbral perceptible (~80ms), frecuentemente porque el batch size es demasiado alto o el hardware no tiene suficiente ancho de

banda de VRAM para el modelo elegido. La solución es reducir el máximo de requests concurrentes o mover a un hardware con mayor ancho de banda de VRAM.

Entender la anatomía de la latencia —prefill vs decode vs red— es el prerrequisito para aplicar las técnicas de optimización que se presentan en las secciones siguientes: speculative decoding para reducir la latencia de decode, Flash Attention para reducir la latencia de prefill, y prefix caching para eliminar el prefill de prefijos compartidos.

Módulo 8 – Capítulo 08 – Sección 02

Especulative decoding: acelerar la generación usando un modelo draft pequeño

La limitación física del decode —que cada token de salida requiere leer todos los pesos del modelo desde VRAM— parece insuperable. Si la física del ancho de banda establece un techo de 140 tokens/s para una request con un modelo de 7B en BF16 en una A100, la única forma de superarlo sería tener una GPU con mayor ancho de banda de VRAM o comprimir más el modelo. Speculative decoding viola esta aparente restricción con un insight algorítmico elegante: en lugar de generar tokens de a uno con el modelo objetivo, genera múltiples tokens candidatos con un modelo draft más pequeño y rápido, y los verifica todos en un único forward pass paralelo del modelo objetivo.

El mecanismo funciona en dos fases en cada iteración. Primero, un modelo draft pequeño (típicamente 1B-3B parámetros, 10-20x más rápido que el modelo objetivo) genera K tokens candidatos en K steps secuenciales de decode —la misma operación que haría el modelo objetivo, pero mucho más rápida por el menor número de parámetros a leer. Segundo, el modelo objetivo evalúa los K tokens candidatos en un **único forward pass paralelo**: en lugar de un único token de entrada como en el decode estándar, el modelo objetivo recibe los K tokens candidatos del draft y calcula simultáneamente la distribución de probabilidad que hubiera producido para cada uno. Si la distribución del modelo objetivo en la posición k es suficientemente similar a la del draft (controlado por un umbral de aceptación), el token se acepta; en el primer rechazo, el modelo objetivo genera el token correcto para esa posición y la secuencia de aceptación termina.

La propiedad matemática crucial es que el proceso de aceptación/rechazo garantiza que la distribución de tokens generados es **idéntica** a la que habría producido el modelo objetivo sin draft. No es una aproximación: si el draft propone el token A y el modelo objetivo hubiera generado con alta probabilidad exactamente ese token A, se acepta; si el draft propone B pero el modelo objetivo asigna alta probabilidad a C, se rechaza B y se genera C. El resultado final es una secuencia de tokens con la misma distribución probabilística que el modelo objetivo sin draft, pero generada más rápidamente cuando el draft predice correctamente.

La **tasa de aceptación del draft** es la métrica clave que determina la aceleración real: si el draft predice correctamente el 80% de los tokens (tasa de aceptación $\alpha = 0.8$) con $K=5$ tokens candidatos, el número esperado de tokens aceptados por iteración es $K \times \alpha / (1 - \alpha^K) \approx 3.4$, obtenidos al costo de un único forward pass del modelo objetivo en lugar de 3.4 pasos de decode individuales. La aceleración efectiva es aproximadamente 2.5-3x en dominios donde el texto es predecible (conversación, código en lenguajes populares). En dominios de alta entropía (texto creativo, generación de datos diversos), la tasa de aceptación puede caer al 50-60%, reduciendo la aceleración a 1.5-2x.

La implementación en vLLM es directa: `vllm serve meta-llama/Llama-3.1-8B-Instruct --speculative-model meta-llama/Llama-3.2-1B-Instruct --num-speculative-tokens 5` activa speculative decoding con Llama 3.2 1B como draft para Llama 3.1 8B. La compatibilidad de vocabulario (mismo tokenizador) es un requisito estricto: modelos de la misma familia con el mismo tokenizador son siempre compatibles como pares objetivo/draft.

Implementaciones y variantes del speculative decoding

- **Draft model selection:** el modelo draft debe ser de la misma familia que el objetivo para máxima tasa de aceptación; mismo tokenizador es requisito estricto; Llama 3.2 1B como draft para Llama 3.1 8B.
- **Medusa (self-speculation):** cabezas adicionales de predicción en el modelo objetivo para $K+1$, $K+2$... tokens en paralelo; sin segundo modelo; aceleración 1.5-2.5x; entrenamiento adicional requerido para las cabezas.
- **Eagle:** utiliza features del último token del modelo objetivo como input del draft; tasas de aceptación superiores a Medusa (0.85-0.95); aceleración 2-3.5x; disponible en vLLM con `--speculative-model Eagle-LLaMA3-Instruct-8B`.
- **Configuración en vLLM:** `--speculative-model <draft> --num-speculative-tokens 5; --speculative-draft-tensor-parallel-size 1` para draft con menos GPUs que el objetivo en configuraciones multi-GPU.
- **Cuándo NO usar speculative decoding:** batch size alto (>16 requests simultáneas); workloads con alta entropía de texto; cuando el overhead del modelo draft supera el ahorro en steps del modelo objetivo.

Nota del Arquitecto: Speculative decoding beneficia más a los escenarios de baja concurrencia (1-4 requests simultáneas) donde el decode es claramente el cuello de botella y la GPU está subutilizada. Con batches grandes, la GPU ya está compute-

bound en el decode del modelo objetivo, y el overhead del modelo draft reduce el throughput agregado en lugar de mejorarlo. Mide siempre el impacto en TTFT-P95 y TBT-P95 con tráfico representativo antes de activar speculative decoding en producción.

Speculative decoding es la técnica más efectiva para reducir la latencia percibida por usuarios individuales en escenarios de baja concurrencia. La sección siguiente presenta Flash Attention, la optimización que reduce la latencia de prefill y el uso de memoria para contextos largos.

Módulo 8 – Capítulo 08 – Sección 03

Flash Attention 2 y 3: reducción de memoria y mejora de velocidad en atención

La operación de atención en un transformer estándar requiere materializar la matriz de atención completa de tamaño $N \times N$ (donde N es la longitud de la secuencia) en VRAM para aplicar el softmax y calcular el producto con los valores V . Para una secuencia de 128K tokens —longitud de contexto soportada por Llama 3.1— esta matriz ocupa $128K \times 128K \times 2 \text{ bytes} = 33 \text{ GB}$, más del 40% de la VRAM de un H100 de 80 GB, empleada solo en el mecanismo de atención de una única capa. Para 32 capas, la atención estándar sería físicamente imposible con contextos largos. Flash Attention resuelve este problema sin cambiar el resultado matemático de la operación.

La idea central de Flash Attention es reorganizar el cómputo de $\text{softmax}(QK^T/\sqrt{d})V$ en un patrón que minimiza las transferencias entre la VRAM (memoria de alta capacidad pero mayor latencia) y el SRAM on-chip de los Streaming Multiprocessors de la GPU (memoria de alta velocidad pero tamaño limitado a $\sim 192 \text{ KB}$ por SM en A100). La implementación estándar lee Q, K, V completos a SRAM, computa la matriz de atención $N \times N$ en SRAM, escribe la matriz $N \times N$ de vuelta a VRAM para el softmax, y luego la lee nuevamente para el producto con V : cuatro transferencias grandes a VRAM por capa. Flash Attention utiliza el **algoritmo de Online Softmax** de Milakov y Gimelshein para dividir Q, K, V en bloques (tiles) que caben en SRAM, computar la atención tile por tile de forma incremental, y nunca materializar la matriz $N \times N$ completa: las transferencias a VRAM se reducen de $O(N^2)$ a $O(N)$ en el número de elementos transferidos.

Flash Attention 2 (2023) añade mejoras de paralelización sobre la implementación original. La versión 1 tenía un problema de distribución del trabajo: en el forward pass, el trabajo se distribuía entre los warps de un SM de forma subóptima, dejando algunos warps idle mientras otros estaban saturados. FA2 redistribuye el trabajo dividiendo los loops de Q (en lugar de K y V) entre los warps, mejorando la ocupancia del SM y reduciendo la sincronización necesaria. El resultado es un speedup adicional de 20-50% sobre FA1 en

operaciones de atención en A100, especialmente en secuencias largas y con cabezas de atención con dimensión grande.

Flash Attention 3 (2024) introduce soporte para FP8 en H100, aprovechando el Transformer Engine de NVIDIA. Con FP8, la operación de atención usa la mitad de la memoria que BF16 y es 2x más rápida en los Tensor Cores de cuarta generación de H100. FA3 también introduce asincronismo entre el pipeline de datos (carga de tiles) y el pipeline de cómputo (multiplicación matricial), solapando la latencia de acceso a SRAM con el cómputo de los tiles anteriores para mayor throughput. El "incoherent processing" de FA3 mejora adicionalmente el throughput en distribuciones de atención muy concentradas (sparse) como las que producen modelos de código donde la atención se concentra en tokens sintácticamente relevantes próximos.

La integración de Flash Attention en el ecosistema de LLMs es transparente para el usuario. Desde PyTorch 2.0, `torch.nn.functional.scaled_dot_product_attention()` usa Flash Attention automáticamente cuando está disponible en el hardware y la precisión es compatible. Hugging Face transformers activa FA2 con `attn_implementation="flash_attention_2"` en la carga del modelo. vLLM usa Flash Attention internamente para todas las operaciones de atención en GPU por defecto. El usuario no necesita activar nada manualmente en la mayoría de los casos.

Aspectos técnicos de Flash Attention

- **Tiling y SRAM:** divide Q, K, V en bloques que caben en el SRAM on-chip (~192 KB por SM en A100); procesa cada bloque completamente antes de escribir el resultado a VRAM; reduce transferencias VRAM de $O(N^2)$ a $O(N)$.
- **Causal masking eficiente:** la máscara triangular de los modelos autoregresivos se implementa sin materializar la máscara completa; solo los tiles de la diagonal requieren masking explícito.
- **Integración en PyTorch y transformers:**
`torch.nn.functional.scaled_dot_product_attention()` usa FA automáticamente;
`attn_implementation="flash_attention_2"` en Hugging Face transformers.
- **Speedup real:** en A100 con secuencias de 4096 tokens, FA2 ofrece 2.5-3x más throughput que la atención estándar; beneficio mayor para secuencias largas.
- **Limitaciones:** no soporta máscaras de atención arbitrarias de forma eficiente; requiere BF16 o FP16 para FA2, FP8 para FA3; no disponible en CPUs ni en GPUs anteriores a Volta.

Nota del Arquitecto: *Flash Attention es una de las pocas optimizaciones en este módulo que tiene cero trade-offs: el resultado matemático es idéntico al de la atención estándar, la latencia es menor, el uso de VRAM es menor. No hay razón para no activarla. Si tu entorno no la activa automáticamente, el flag `attn_implementation="flash_attention_2"` en Hugging Face transformers y la imagen `vllm/vllm-openai:latest` que incluye FA2 precompilada son las formas más directas de asegurar que está activa.*

Flash Attention resuelve el problema de la atención cuadrática que limita el tamaño de contexto práctico de los LLMs. La sección siguiente aborda el prompt caching: la técnica complementaria que elimina completamente el compute de prefill para los prefijos de prompt que se repiten entre peticiones.

Módulo 8 – Capítulo 08 – Sección 04

Prompt caching en servidores locales: reutilización del KV cache entre peticiones

Flash Attention reduce el costo de computar el KV cache una vez para un prompt dado. El prefix caching elimina la necesidad de computar ese KV cache en absoluto cuando el mismo prompt ya fue procesado por una petición anterior: reutiliza los bloques de KV cache almacenados de peticiones previas que comparten el mismo prefijo, reduciendo el TTFT en el porcentaje del contexto que se puede cachear.

El caso de uso más directo y de mayor impacto es la aplicación multi-usuario con un system prompt estático largo. Considera un asistente corporativo donde el system prompt de 1.500 tokens describe las políticas de la empresa, los productos disponibles, el tono de comunicación y las restricciones de respuesta. Sin prefix caching, cada una de las 10.000 peticiones diarias del asistente debe procesar esos 1.500 tokens de system prompt en la fase de prefill, incluso cuando el contenido es idéntico en todas las peticiones. Con prefix caching activo en vLLM, la primera petición del día computa el KV cache del system prompt y lo almacena en el pool de PagedAttention. Todas las peticiones posteriores que comienzan con ese mismo system prompt detectan el cache hit y saltan directamente al prefill de los tokens exclusivos de cada petición (la pregunta del usuario), eliminando el 75% o más del trabajo de prefill si el system prompt representa esa fracción del prompt total.

vLLM implementa prefix caching con el flag `--enable-prefix-caching`. El mecanismo interno usa un **Radix Tree** para gestionar los bloques de KV cache compartibles: cada nodo del árbol representa un bloque de 16 tokens; peticiones con el mismo prefijo comparten los mismos nodos hasta el punto donde sus contenidos divergen. El cache hit solo ocurre cuando los primeros K tokens de la nueva petición son **byte-a-byte idénticos** a los de una petición previa, donde K es múltiplo del block_size de 16 tokens. Una diferencia de un espacio, una mayúscula distinta o un carácter de puntuación diferente en el system prompt produce un miss completo. Por esta razón, en aplicaciones con prefix caching activo, el system prompt debe ser gestionado como un artefacto versionado: una cadena exacta con su codificación UTF-8, no generada dinámicamente en cada petición.

El **cache warming** al inicio del servidor es una práctica que amplifica el beneficio del prefix caching en producción. Al iniciar el servidor de vLLM, enviar inmediatamente una petición de warmup con el system prompt completo pre-computa y almacena el KV cache de ese prefijo antes de que lleguen requests reales de usuarios. Sin warmup, las primeras N peticiones del día —donde N es el número de peticiones necesarias para llenar la capacidad del cache pool con el prefijo compartido— pagan el costo de prefill completo. Con warmup, la primera petición real ya tiene el cache del system prompt disponible.

llama.cpp también soporta un forma de caching de contexto orientada a sesiones: `--save-cache-kv ruta/cache.bin` serializa el estado del KV cache a disco después de procesar un prompt largo; `--load-cache-kv ruta/cache.bin` lo restaura en la siguiente sesión. Este mecanismo es especialmente útil para pipelines RAG donde el contexto de recuperación —10-50 documentos recuperados, posiblemente miles de tokens— se actualiza infrecuentemente (cada hora, cada día) pero se usa en todas las peticiones de ese periodo. Guardar el KV cache del contexto de recuperación evita recomputarlo en cada petición, reduciendo el TTFT de los segundos que podría tomar prefill de 4K-8K tokens de documentos a los milisegundos del decode de los tokens de la query.

Implementaciones y consideraciones del prefix caching

- **Requisitos para el cache hit:** los primeros K tokens deben ser byte-a-byte idénticos, con K múltiplo del block_size de 16; cualquier diferencia (espacio, capitalización) produce miss completo.
- **Radix Tree de vLLM:** gestiona bloques de KV cache compartibles entre requests; compartición eficiente de prefijos de distintas longitudes; liberación de nodos cuando las requests completan.
- **LLM caching a nivel de aplicación:** semantic cache con Redis o GPTCache para queries semánticamente similares; complementario al prefix caching; opera a nivel de respuesta completa, no de KV cache.
- **Cache warming:** enviar request de warmup al iniciar el servidor; pre-computa el KV del system prompt antes de recibir requests reales; especialmente importante después de reinicios planificados.
- **llama.cpp KV cache persistence:** `--save-cache-kv` / `--load-cache-kv` para pipelines RAG con contextos de recuperación que se actualizan infrecuentemente.

Nota del Arquitecto: El prefix caching es la optimización más fácil de implementar con el mayor retorno de inversión para aplicaciones enterprise: activar `--enable-prefix-caching` en vLLM y asegurar que el system prompt sea idéntico entre peticiones (sin timestamps, sin IDs de sesión dinámicos en el system prompt) puede reducir el costo de prefill en un 70-80% para aplicaciones donde el system prompt representa más del 50% del prompt total. El esfuerzo de implementación es literalmente un flag en el comando de inicio del servidor.

El prefix caching cierra el ciclo de optimizaciones de latencia y costo para las fases de prefill. La sección siguiente presenta las herramientas de perfilado que permiten identificar cuál de estas técnicas ofrece el mayor beneficio en el sistema específico de producción.

Módulo 8 – Capítulo 08 – Sección 05

Perfilado de inferencia: identificar cuellos de botella con herramientas de profiling

Conocer las técnicas de optimización disponibles no es suficiente si no se sabe cuál aplicar primero en un sistema específico. Un sistema con alta latencia de TTFT puede estar sufriendo de prompts largos sin chunked prefill, de un modelo draft incorrecto para el dominio que reduce la tasa de aceptación en speculative decoding, de un pool de KV cache subutilizado, o simplemente de que la GPU elegida no tiene suficiente ancho de banda para el throughput requerido. Aplicar la optimización incorrecta no solo no ayuda: puede empeorar la situación al añadir overhead sin abordar la causa raíz. El perfilado es el proceso de medir para saber dónde está realmente el cuello de botella.

El perfilado de inferencia de LLMs opera en múltiples niveles simultáneos con herramientas distintas para cada nivel. El nivel más alto —el más útil para el monitoreo continuo en producción— es el endpoint `/metrics` de vLLM, que expone métricas Prometheus actualizadas en tiempo real. Las métricas más importantes son: `vllm:num_requests_running` y `vllm:num_requests_waiting` (para ver si hay backpressure en la cola), `vllm:gpu_cache_usage_perc` (porcentaje de uso del pool de KV cache; si supera el 90% consistentemente, el pool está demasiado pequeño), `vllm:time_to_first_token_seconds` (histograma de TTFT), y `vllm:time_per_output_token_seconds` (histograma de TBT). Configurar un dashboard de Grafana con estas métricas y alertas sobre los P95 de TTFT y TBT es el primer paso del monitoreo de producción y debe estar disponible antes del primer despliegue, no después del primer incidente.

El nivel GPU del perfilado requiere herramientas de mayor resolución cuando el dashboard de Prometheus indica anomalías pero no revela la causa. `nvidia-smi dmon -s puct -d 1` proporciona monitoreo en tiempo real a 1 Hz de utilización de GPU (%), uso de VRAM (MB), temperatura (°C) y potencia (W): suficiente para detectar problemas estructurales como GPU constantemente al 30% de utilización (indica throttling por temperatura o instancia sobredimensionada) o VRAM al 95%+ (indica pool de KV cache al límite). Para diagnóstico más profundo, **NVIDIA Nsight Systems** (`nsys profile`) captura una traza temporal

completa de todos los kernels CUDA, mostrando exactamente qué operaciones dominan el tiempo de ejecución y si hay periodos de GPU idle entre kernels que indican scheduling overhead.

Para diagnóstico de cuellos de botella en el código Python del motor de serving (tokenización, serialización de respuestas, middleware), `py-spy` proporciona profiling de CPU sin instrumentación del código: `py-spy top --pid <pid>` muestra en tiempo real las funciones Python con mayor uso de CPU, permitiendo identificar si el cuello de botella está en la GPU o en el procesamiento Python del servidor. Este diagnóstico es especialmente relevante cuando el `nvidia-smi` muestra utilización GPU baja pero las métricas de Prometheus muestran TTFT alto: indica que el tiempo se está gastando en Python (serialización del JSON, tokenización, lógica del servidor) en lugar de en la GPU.

Para benchmarking end-to-end antes del despliegue, los scripts `benchmark_throughput.py` y `benchmark_latency.py` incluidos en el repositorio de vLLM permiten generar workloads sintéticos configurables con longitudes de prompt y output variables, midiendo TTFT P50/P95/P99 y throughput bajo distintos niveles de carga concurrente. Este benchmarking pre-despliegue debe ejecutarse con parámetros que reflejen el percentil 95 del tráfico real de producción —longitudes de prompt, distribución de output lengths, número de requests concurrentes— no con los mejores casos.

Herramientas de perfilado por nivel

- **Dashboard Prometheus + Grafana:** métricas en tiempo real de vLLM (`num_requests_running`, `gpu_cache_usage_perc`, `time_to_first_token_seconds`); primer nivel de monitoreo de producción; alertas sobre P95 de TTFT y TBT.
- `nvidia-smi dmon`: monitoreo a 1 Hz de GPU utilization, VRAM, temperatura y potencia; detección de throttling, subutilización y saturación de VRAM.
- **Nsight Systems:** traza completa de kernels CUDA con timeline visual; identifica GPU idle, solapamiento de kernels y operaciones dominantes; `nsys profile --trace=cuda,nvtx python -m vllm.entrypoints.openai.api_server`.
- `py-spy`: profiling de CPU Python sin instrumentación; `py-spy top --pid <pid>` muestra funciones Python con mayor uso de CPU; diagnóstica cuellos de botella en el servidor vs en la GPU.
- **Scripts de benchmark de vLLM:** `benchmark_throughput.py` y `benchmark_latency.py`; workloads sintéticos configurables; TTFT P50/P95/P99 y throughput bajo carga concurrente.

Nota del Arquitecto: El orden correcto de perfilado es de afuera hacia adentro: primero el dashboard de Grafana con métricas de Prometheus para identificar el síntoma (TTFT alto, queue depth, baja utilización de GPU); luego `nvidia-smi` para ver el nivel de utilización real de la GPU; solo si el problema no está claro con esas dos herramientas, Nsight Systems o `py-spy` para el diagnóstico profundo. El 80% de los problemas de producción son visibles con las métricas de Prometheus y `nvidia-smi`; Nsight Systems es para el 20% restante.

El perfilado riguroso es la base del ciclo de mejora continua del rendimiento de inferencia. La sección de cierre sintetiza las técnicas de optimización del capítulo como una disciplina de sistemas que requiere comprensión de hardware además de modelos.

Módulo 8 – Capítulo 08 – Sección 06

Cierre: la optimización de inferencia local es una disciplina de sistemas, no solo ML

Las técnicas presentadas en este capítulo —Flash Attention, speculative decoding, prefix caching, chunked prefill, y el perfilado con Nsight y Prometheus— son fundamentalmente innovaciones de sistemas de computación aplicadas al dominio de los LLMs. Requieren comprensión de la jerarquía de memoria de las GPUs (HBM vs SRAM on-chip), del perfil de operaciones de los transformers (memory-bound vs compute-bound), del scheduling de requests en sistemas concurrentes (continuous batching, KV cache management), y de la teoría de colas (queue depth, percentiles de latencia). Un AI Engineer que solo comprende los aspectos de modelo —arquitectura, benchmarks, cuantización— pero no los aspectos de sistemas, tomará decisiones de optimización subóptimas porque atacará los síntomas visibles en lugar de las causas raíz.

La brecha de rendimiento entre una implementación naiva y una optimizada del mismo modelo es frecuentemente de un orden de magnitud. Considerar Llama 3 8B en BF16 sirviendo usuarios en producción: con Hugging Face Transformers en modo básico sin batching, el throughput puede ser de 100-200 tokens/s por GPU y el TTFT de 500ms-2s para prompts moderados; con vLLM + PagedAttention + continuous batching + prefix caching + Flash Attention 2 + chunked prefill, el mismo modelo en el mismo hardware puede superar 2.500 tokens/s de throughput agregado y TTFT < 200ms para el P95 bajo carga moderada. Esa diferencia de 10-15x no proviene de ningún cambio en el modelo: proviene de cómo se gestiona la memoria, cómo se planifican las requests y cómo se implementa matemáticamente la operación de atención.

La optimización de inferencia tampoco es un proyecto único de mejora sino un proceso continuo. Los modelos se actualizan (nuevas versiones de Llama, Mistral, Qwen con diferentes tamaños de KV cache o ventanas de atención), el tráfico del producto evoluciona (mayores contextos promedio a medida que los usuarios aprenden a usar el producto), y el hardware se renueva (nuevas generaciones de GPU con FP8 nativo o mayor ancho de banda

de VRAM). Cada uno de estos cambios puede desplazar el cuello de botella a un componente distinto y requerir re-perfilar para aplicar la optimización correcta en el nuevo contexto.

Lo que distingue al AI Engineer maduro en este dominio es la capacidad de leer las métricas de Prometheus, identificar el componente limitante del sistema actual, seleccionar la técnica de optimización correspondiente, medirla después de aplicarla, y iterar. Esta disciplina de optimización empírica guiada por datos, aplicada a sistemas de serving de LLMs, es exactamente la misma que los ingenieros de bases de datos aplican con EXPLAIN ANALYZE en SQL o los ingenieros de sistemas aplican con perf en C++: el dominio es nuevo, el método es el mismo.

Idea central

Optimizar la inferencia de un LLM local no termina con elegir el modelo y la cuantización correctos: Flash Attention, speculative decoding, prefix caching y el motor de serving correcto son multiplicadores de rendimiento que juntos pueden reducir el costo por token 5-10x sobre la misma GPU.

"The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise." — Edsger W. Dijkstra, padre de los algoritmos modernos, recordando que las abstracciones de los motores de inferencia (PagedAttention, Flash Attention) no ocultan la complejidad sino que crean el nivel correcto donde la optimización puede ser exacta y medible.